

GRAND JAVA HOTEL MANAGEMENT SYSTEM

Build Your Own Project — VITyarthi

Subject: Java Programming / Object-Oriented Programming

Project Type: Console-based Java application with CSV persistence

Name:Sagan Gangwar

Reg No:25BAI10057

Introduction

The Grand Java Hotel Management System is a modular console application designed to digitize common hotel front-desk activities. It manages rooms, guests, reservations, cancellation, billing and a dashboard. The system demonstrates Java programming concepts in a real-world context.

1. Problem Statement

Manual hotel records can cause duplicate reservations, incorrect room status, repeated data entry and billing mistakes. The proposed system provides a structured workflow for storing guest details, checking room availability, creating reservations and generating bills.

2. Objectives

- Apply Java classes, objects, constructors and methods.
- Demonstrate encapsulation and modular design.
- Use collections for in-memory records.
- Use file handling for persistent CSV storage.
- Validate user input and handle common errors.
- Produce a practical, demonstrable academic project.

3. Functional Requirements

The system contains more than the required three major functional modules:

1. Room Management — display rooms and track availability.
2. Guest Management — register and view guests.
3. Booking Management — create and cancel reservations.
4. Billing — calculate room charges, service charges and tax.
5. Reporting — show room counts and occupancy rate.
6. Persistence — save and reload records using CSV files.

4. Non-Functional Requirements

- Performance: menu operations should complete quickly for academic-scale data.
- Usability: numbered menus and clear prompts are used.
- Reliability: invalid numeric input is caught and retried.
- Maintainability: responsibilities are divided into model and manager classes.
- Resource efficiency: lightweight CSV storage avoids a database server.
- Error handling: file and input exceptions are handled with user-friendly messages.

5. System Architecture

The application follows a simple layered architecture: Console/UI → Manager classes → Model classes → FileManager → CSV files. Main coordinates user interaction; manager classes contain business operations; model classes represent data; FileManager centralizes persistence.

6. Design Diagrams

The repository contains Mermaid source files for the architecture, use-case, workflow, sequence and class diagrams. They can be rendered in GitHub or a Mermaid-compatible editor. An ER diagram is not required because the project deliberately uses file storage rather than a relational database.

7. Design Decisions & Rationale

CSV persistence was selected so that the project can run on a standard JDK without external database installation. Separate manager classes make the code easier to understand and maintain. InputValidator keeps validation logic separate from business logic. A 12% tax is used consistently for the academic billing demonstration.

8. Implementation Details

The project contains 13 Java classes. Room, Guest, Booking, Bill and Staff are model classes. RoomManager, GuestManager, BookingManager and BillManager perform domain operations. FileManager handles CSV read/write. ReportManager produces dashboard information. InputValidator validates interactive input. Main provides the console workflow.

9. Screenshots / Results

The application prints a main menu, room list, guest records, booking confirmations, cancellation messages, bills and dashboard statistics. The repository can be run to capture actual screenshots for the final report.

10. Testing Approach

Testing is based on functional test cases covering valid operations, invalid menu selections, invalid numeric input, occupied-room booking, booking cancellation, billing and date validation. The full test table is provided in docs/TestCases.md.

11. Challenges Faced

Key implementation challenges include coordinating room availability with booking creation/cancellation, keeping generated IDs unique during a run, persisting records after changes and validating interactive console input without terminating the program.

12. Learnings & Key Takeaways

The project reinforces object-oriented design, encapsulation, constructors, methods, collections, file handling, exception handling, control flow and modular programming. It also introduces software engineering practices such as requirements analysis, diagrams, testing, documentation and Git version control.

13. Future Enhancements

- Login and role-based access.
- GUI using JavaFX or Swing.
- MySQL database with JDBC.
- Multiple-room and multi-guest reservations.
- Date-overlap validation for future bookings.
- Payment modes and printable invoices.
- Advanced revenue and occupancy analytics.
- Unit tests using JUnit.

14. References

- Course/VITyarthi Build Your Own Project submission guidelines supplied for this project.
- Java SE documentation and standard Java library concepts used in the implementation.

Appendix A — Run Instructions

```
javac -d out src/hotelmanagement/*.java
java -cp out hotelmanagement.Main
```

Appendix B — Suggested Demonstration Flow

1. Display rooms → 2. Add guest → 3. Create booking → 4. Show room as occupied → 5. Generate bill → 6. View dashboard → 7. Cancel booking → 8. Show room available again → 9. Exit.